

Aula 7 — Autenticação e Autorização

Módulo: Razor Pages e UI Server-Side

Carga horária: 4 horas

Projeto base: Aula01.Web

Objetivo da aula:

Implementar **autenticação e autorização profissional** usando **ASP.NET Core Identity**, compreender as diferenças entre **Cookies e JWT**, trabalhar com **Roles e Claims**, e **proteger páginas e funcionalidades** de acordo com o perfil do usuário.

PARTE 1 — TEORIA (≈ 2 horas)

1 ASP.NET Core Identity

◆ O que é?

Framework nativo do .NET para:

- Autenticação
- Autorização
- Gestão de usuários
- Hash de senhas
- Roles e Claims

Inclui:

- UserManager
- SignInManager
- RoleManager

👉 Padrão de mercado para aplicações server-side.

2 Cookies vs JWT

◆ Cookies (Identity)

- ✓ Sessão server-side
- ✓ Mais simples
- ✓ Ideal para Razor Pages / MVC

Fluxo:

Login → Cookie → Request autenticada

◆ JWT (API / SPA)

- ✓ Stateless
- ✓ Ideal para React / Blazor WASM
- ✓ Escalável

Fluxo:

Login → Token JWT → Header Authorization

👉 Nesta aula: **Cookies (Identity)**

JWT será aprofundado no **Módulo React/Blazor WASM**.

3 Roles e Claims

◆ Roles

Perfis de acesso:

- Admin
- Gerente
- Usuário

```
[Authorize(Roles = "Admin")]
```

◆ Claims

Atributos do usuário:

- Email
- Departamento
- Permissões customizadas

```
User.HasClaim("Departamento", "TI")
```

4 Segurança em Aplicações Web

Boas práticas:

- Hash de senha (bcrypt)
 - HTTPS
 - Anti-forgery token
 - Não expor mensagens sensíveis
 - Princípio do menor privilégio
-

PARTE 2 — PRÁTICA (≈ 2 horas)

Objetivo Prático

Adicionar **autenticação completa** ao sistema administrativo:

- Login
 - Registro
 - Roles
 - Proteção de páginas Razor
-

1 Instalação dos Pacotes Identity

```
dotnet add package Microsoft.AspNetCore.Identity.EntityFrameworkCore dotnet add package Microsoft.AspNetCore.Identity.UI
```

2 Criando o DbContext com Identity

 Data/AppDbContext.cs

```
using Microsoft.AspNetCore.Identity.EntityFrameworkCore; using Microsoft.EntityFrameworkCore; namespace Aula01.Web.Data { //
IdentityDbContext já inclui as tabelas de autenticação public class AppDbContext : IdentityDbContext { public
AppDbContext(DbContextOptions<AppDbContext> options) : base(options) { } // Demais DbSets do sistema public DbSet<Produto>
Produtos { get; set; } public DbSet<Categoria> Categorias { get; set; } } }
```

3 Configurando Identity no Program.cs

```
builder.Services .AddIdentity<IdentityUser, IdentityRole>() .AddEntityFrameworkStores<AppDbContext>()
.AddDefaultTokenProviders(); builder.Services.ConfigureApplicationCookie(options => { options.LoginPath = "/Account/Login";
```

```
options.AccessDeniedPath = "/Account/AcessoNegado"; });
```

4 Criando as Migrations do Identity

```
dotnet ef migrations add IdentityInitial dotnet ef database update
```

✓ Cria tabelas:

AspNetUsers , AspNetRoles , AspNetUserRoles , etc.

5 Página de Registro (Register)

 Pages/Account/Register.cshtml.cs

```
using Microsoft.AspNetCore.Identity; using Microsoft.AspNetCore.Mvc; using Microsoft.AspNetCore.Mvc.RazorPages; namespace
Aula01.Web.Pages.Account { public class RegisterModel : PageModel { private readonly UserManager<IdentityUser> _userManager;
public RegisterModel(UserManager<IdentityUser> userManager) { _userManager = userManager; } [BindProperty] public
RegisterViewModel Input { get; set; } public async Task<IActionResult> OnPostAsync() { if (!ModelState.IsValid) return Page();
var user = new IdentityUser { Username = Input.Email, Email = Input.Email }; var result = await _userManager.CreateAsync(user,
Input.Password); if (result.Succeeded) { await _userManager.AddToRoleAsync(user, "Usuario"); return RedirectToPage("Login"); }
foreach (var error in result.Errors) ModelState.AddModelError("", error.Description); return Page(); } } }
```

 RegisterViewModel.cs

```
using System.ComponentModel.DataAnnotations; public class RegisterViewModel { [Required, EmailAddress] public string Email { get;
set; } [Required, MinLength(6)] public string Password { get; set; } }
```

6 Página de Login

 Pages/Account/Login.cshtml.cs

```
using Microsoft.AspNetCore.Identity; using Microsoft.AspNetCore.Mvc; using Microsoft.AspNetCore.Mvc.RazorPages; namespace
Aula01.Web.Pages.Account { public class LoginModel : PageModel { private readonly SignInManager<IdentityUser> _signInManager;
public LoginModel(SignInManager<IdentityUser> signInManager) { _signInManager = signInManager; } [BindProperty] public
LoginViewModel Input { get; set; } public async Task<IActionResult> OnPostAsync() { if (!ModelState.IsValid) return Page(); var
result = await _signInManager.PasswordSignInAsync( Input.Email, Input.Password, false, false); if (result.Succeeded) return
RedirectToPage("/Produtos/Index"); ModelState.AddModelError("", "Login inválido"); return Page(); } } }
```

7 Protegendo Páginas com [Authorize]

◆ Proteção global

```
builder.Services.AddRazorPages(options => { options.Conventions.AuthorizeFolder("/Produtos"); });
```

◆ Proteção por Role

```
[Authorize(Roles = "Admin")] public class CreateModel : PageModel
```

8 Exibindo Conteúdo por Perfil (UI)

```
@if (User.IsInRole("Admin")) { <a asp-page="/Produtos/Create">Novo Produto</a> }
```

Criando Roles Automaticamente (Seed)

 Data/IdentitySeeder.cs

```
using Microsoft.AspNetCore.Identity; public static class IdentitySeeder { public static async Task SeedAsync(IServiceProvider services) { var roleManager = services.GetRequiredService<RoleManager<IdentityRole>>(); string[] roles = { "Admin", "Usuario" }; foreach (var role in roles) { if (!await roleManager.RoleExistsAsync(role)) await roleManager.CreateAsync(new IdentityRole(role)); } } }
```

 Chamar no Program.cs :

```
using (var scope = app.Services.CreateScope()) { await IdentitySeeder.SeedAsync(scope.ServiceProvider); }
```

Resultado Esperado

-  Autenticação completa
-  Controle por perfil
-  Páginas protegidas
-  UI condicional por Role
-  Segurança alinhada ao mercado

Atividade Avaliativa

 Desafio prático:

1. Criar role Gerente
2. Restringir edição de produtos
3. Criar Claim Departamento
4. Exibir Claim no layout